261-406—ภาคเรียนที่ 1 ปีการศึกษา 2569
Source language specifications

In our course's project, you will implement a compiler for *CPE* (recursive acronym for *CPE: Programming Extraordinaire*), a lite version of a functional language. This document aims to provide an overview of language features that should be implemented. You are welcome—and even encouraged—to raise issues and discuss about the features on Ed. Major changes from such issues and discussions will appear here.

0 Updates since release

- none yet

1 Overview of the language

A program in this language will be one or more source files containing definitions for expressions and functions. One of the functions, named `main`, is designated to be the entry point to the program. The `main` function takes a single argument, which is always a list of strings, and always returns a string. The argument can be thought of as the input to the program.

A source file may import additional source files that define additional functions using the **`import`** statement. Additional source files should be in the same directory as the primary source file. Every function in each additional source file will be imported, but each of them may be prefixed with a *namespace* that will be specified along with the **`import`** statement to avoid name collisions. It is a compile-time error if multiple functions of the same name (including `main`) are imported into the same namespace. Hence, files containing `main` should always be imported with a namespace. Still, the `main` function could be absent in a source file that will be primarily imported and not run directly.

1.1 Example programs

Here are a few examples of programs that illustrate language features. Details about each feature are explained in the rest of the specifications.

Hello, world! This program simply ignores the argument and returns `Hello, world!` as the result.

Listing 1: `hello_world.cpe`

```
def main (arg : [String]) : String =  # ignores input
  "Hello, world!"
```

Common functions This file defines commonly used functions in functional programming.

Listing 2: `pervasives.cpe`

```
def map (f : a -> b) (xs : [a]) : [b] =
  match xs with
  | [] -> []
  | hd::tl -> f hd :: map f tl

def filter (p : a -> Bool) (xs : [a]) : [a] =
  match xs with
  | [] -> []
```

```
  | hd::tl -> if p hd then hd :: filter p tl else filter p tl

def foldl (accum : b -> a -> b) (init : b) (xs : [a]) : b =
  match xs with
  | [] -> init
  | hd::tl -> foldl accum (accum init hd) tl

def foldr (accum : a -> b -> b) (init : b) (xs : [a]) : b =
  match xs with
  | [] -> init
  | hd::tl -> accum hd (foldr accum init tl)
```

Utility functions This file defines utility functions for converting strings to and from various types.

Listing 3: `utils.cpe`

```
import pervasives

def reverse (xs : [a]) : [a] =
  foldl (\ (acc : [a]) (x : a) : [a] => x::acc) [] xs

def takeWhile (p : a -> Bool) (xs : [a]) : [a] =
  match xs with
  | [] -> []
  | hd::tl -> if p hd then hd :: takeWhile p tl else []

def dropWhile (p : a -> Bool) (xs : [a]) : [a] =
  match xs with
  | [] -> []
  | hd::tl -> if p hd then dropWhile p tl else xs

def span (p : a -> Bool) (xs : [a]) : ([a], [a]) =
  match xs with
  | [] -> ([], [])
  | hd::tl -> if p hd
              then match span p tl with
                   | (fr, bk) -> (hd::fr, bk)
              else ([], xs)

def not (b : Bool) : Bool = match b with
  | False -> True
  | True -> False

def break (p : a -> Bool) (xs : [a]) : ([a], [a]) =
  span (not . p) xs

def isSpace (c : Char) : Bool =
  c == ' ' || c == '\n' || c == '\t'

def words (str : String) : [String] =
  match dropWhile isSpace str with
  | "" -> []
  | s -> match break isSpace s with
         | (w, rest) -> w :: words rest

def unwords (strs : [String]) : String =
  match strs with
```

```
  | [] -> ""
  | hd::[] -> hd
  | hd::tl -> hd ++ " " ++ unwords tl

def charToDigit (c : Char) : Integer =
  let a = ord c - 48
  in if a >= 0 && a <= 9 then a else error "not a number"

def digitToChar (a : Integer) : Char =
  if a >= 0 && a <= 9 then chr (a+48) else error "not a digit"

def parseInt (str : String) : Integer =
  match str with
  | [] -> error "not a number"
  | hd::tl -> if hd == '-'
        then 0 - parseInt tl
        else foldl (\ (acc : Integer) (d : Integer) : Integer => 10 * acc + d)
                0 (map charToDigit str)

def showInt (x : Integer) : String =
  if x == 0 then "0"
  else if x < 0 then '-' :: showInt (-x)
  else
    let intToDigits (x : Integer) : [Integer] =
          if x == 0 then []
          else x % 10 :: intToDigits (x / 10)
    in map digitToChar . reverse . intToDigits $ x

def parseIntList (str : [String]) : [Integer] =
  map parseInt str

def showIntList (xs : [Integer]) : String =
  unwords . map showInt $ xs
```

Addition This program takes two integers and returns their sum.

Listing 4: `add.cpe`

```
import utils

def main (arg : [String]) : String =
  match parseIntList arg with
  | a::b::[] -> showInt (a + b)
  | _ -> error "expected 2 integers"
```

Sorting This program takes a list of integers and returns the list sorted in ascending order.

Listing 5: `sort.cpe`

```
import utils

def sort (xs : [Integer]) : [Integer] =
  match xs with
  | [] -> []
  | x::xs ->
      match partition x xs with
      | (lt, ge) -> (sort lt) ++ (x::[]) ++ (sort ge)
```

```
def partition (pivot : Integer) (xs : [Integer]) : ([Integer], [Integer]) =
  match xs with
  | [] -> ([], [])
  | x::xs ->
      match partition pivot xs with
      | (lt, ge) -> if x < pivot then (x::lt, ge) else (lt, x::ge)

def main (arg : [String]) : String =
  showIntList . sort . parseIntList $ arg
```

User-defined algebraic data types Programmers can define their own algebraic data types, e.g., binary trees.

Listing 6: `tree.cpe`

```
data Tree a = Leaf | Node a (Tree a) (Tree a)

def preorder (t : Tree a) : [a] =
  match t with
  | Leaf -> []
  | Node v l r -> v :: preorder l ++ preorder r

def inorder (t : Tree a) : [a] =
  match t with
  | Leaf -> []
  | Node v l r -> inorder l ++ v::[] ++ inorder r

def postorder (t : Tree a) : [a] =
  match t with
  | Leaf -> []
  | Node v l r -> postorder l ++ postorder r ++ v::[]
```

Certain types and functions are built-in and can be described approximately as follows:

```
data Bool = False | True  # boolean type
data () = ()  # unit type
data (a, b) = (a, b)  # polymorphic pair type
data [a] = [] | a :: [a]  # polymorphic list type
type String = [Char]  # type alias

# list concatenation (left-associative)
def (++) (xs : [a]) (ys : [a]) : [a] =
  match xs with
  | [] -> ys
  | hd::tl -> hd::(tl++ys)

# function composition (right-associative)
def (.) (g : b -> c) (f : a -> b) (x : a) : c = g (f x)

# right-associative function application
def ($) (f : a -> b) (x : a) : b = f x

# ASCII value of a character
def ord (c : Char) : Integer = ...

# character of an ASCII value
def chr (a : Integer) : Char = ...
```

```
24
25 # display error and terminate program
26 def error (msg : String) : a = ...
```

2 Functions

Each top-level function starts with the **def** keyword, followed by its name (starting with a lowercase letter) and one or more identifiers for the formal parameters of the function. Each parameter is tagged with its type. After the formal parameters come the return type of the function and its body, which is an expression.

The names of a function and its parameters are identifiers starting with a lowercase letter. A compile-time error occurs if one attempts to name them starting with an uppercase letter.

Top-level functions may be mutually recursive, as in the following example:

```
1 def isEven (x : Integer) : Bool =
2   if x == 0 then True else if x < 0 then isEven (-x) else isOdd (x-1)
3
4 def isOdd (x : Integer) : Bool =
5   if x == 0 then False else if x < 0 then isOdd (-x) else isEven (x-1)
```

It is a compile-time error if more than one formal parameter of a function have the same name.

3 Variables

Each top-level variable is like a top-level function, but does not have the formal parameter section.

Top-level variables cannot contain circular definitions. In fact, the definition of a variable can only refer to variables already defined earlier in the file. In other words, forward references are not permitted. For example, the following code should not compile:

```
1 def x : Integer = y + 1  # illegal: forward reference
2 def y : Integer = x - 1
```

Top-level functions and variables belong to the same namespace; it is a compile-time error if two functions or variables in a file have the same name.

4 Data types

4.1 Built-in types

Atomic built-in types These consist of booleans (`Bool`), integers (`Integer`), floating-point numbers (`Double`), and characters (`Char`).

Unit type The *unit type* (`()`) is a type that has only one member, namely, *unit* (`()`). This type can be useful when implementing a constant function.

Polymorphic pairs Given any two types `a` and `b`, we can construct a polymorphic pair type `(a, b)`, whose values are pairs in which the first component is a value of type `a` and the second component is a value of type of type `b`.

Polymorphic lists Given a type `a`, we can construct a polymorphic list type `[a]`. List values can be constructed in two ways: the empty list (`[]`) and a *cons* (`::`) of a value of type `a` to an already-existing list of type `[a]`.

4.2 Function types

A function type consists of two or more types or type variables, separated with `->`. The leftmost type (variable) corresponds to the first argument of the function, and the rightmost type (variable) corresponds to the return value of the function.

The `->` operator is right-associative. For example, type `a -> b -> c` has the same meaning as type `a -> (b -> c)`. Parentheses can be used to override the nesting.

4.3 User-defined types

Programmers may define their own types. A user-defined data type declaration begins with the **`data`** keyword, followed by the type declaration, a `=` symbol, and the list of *constructor* declarations. Constructor declarations are separated by a `|`.

Type names are identifiers that begin with an uppercase letter. A compile-time error occurs if one attempts to define a type name with a lowercase letter.

A constructor declaration consists of the name of the constructor, which must be an identifier beginning with an uppercase letter. A constructor may have one or more parameters. In this case, the type of each constructor argument is written after the name.

User-defined types, along with their constructors, may be polymorphic. After the type name and before the `=`, zero or more *type variables* can follow. Each type variable is an identifier that must begin with a lowercase letter, or a compile-time error occurs. In the constructor declarations that follow, the types of constructor arguments may be replaced with type variables. Here are examples:

```
data IntList = EmptyIntList | ConsIntList Int IntList
data List a = Empty | Cons a (List a)

data Either a b = Left a | Right b
data Logged a = Log String a
```

A compile-time error occurs if a type variable appearing in a constructor declaration is not declared in the type declaration.

A compile-time error occurs if type or constructor names are duplicates across all the unprefixed imported files. However, type names and constructor names belong to separate namespaces. Therefore, the following declaration is permitted:

```
data Box a = Box a
```

This declaration creates a polymorphic `Box` type, whose values can be constructed with the `Box` constructor that takes an argument of an appropriate type.

User-defined data types may be mutually recursive, as in the following example:

```
data Even = Zero | ESucc Odd
data Odd = OSucc Even
```

4.4 Type synonyms

Programmers may define *type synonyms* for their convenience. A type synonym declaration begins with the **`type`** keyword, followed by the alias declaration, a `=` symbol, and the underlying type.

Type synonyms may be polymorphic, as in this example:

```
data Map domain codomain = ...
type Endomap s = Map s s
```

All type arguments in the underlying type must appear as type parameters in the alias, or a compile-time error occurs.

5 Source file structure

A source file begins with optional **`import`** statements, followed by one or more definitions of functions, data types, and type synonyms. Each line may end with an *end-of-line comment*, which begins with the `#` directive. *Multi-line comments* are permitted; each is enclosed between `(*` and `*)`. Nesting of multi-line comments is not permitted, e.g., `(*(* comment *)*)` is an invalid comment.

5.1 `import` statements

An **`import`** statement has two possible forms:

- **`import`** `<file-name>`: This statement imports declarations in the specified file into the top-level namespace. If the imported file contains names already in use, a compile-time error occurs.
- **`import`** `<file-name>` **`as`** `<namespace>`: This statement imports declarations like in the first form, but each use of a name declared in the specified file must be prefixed with the specified namespace, separating both names with an exclamation mark (`!`). For example, suppose a source file `imported.cpe` has a declaration named `fun`, and is imported as **`import`** `imported` **`as`** `lib`, then each use of `fun` should be written as `lib!fun`.

6 Expressions

6.1 Literals

Literals represent fixed values that cannot be modified. Values in atomic built-in types may be provided as literals.

- Boolean literals are either `True` or `False`.
- Integer literals are nonnegative integers between 0 and $2^{62} - 1$, inclusive.
- Floating-point literals are nonnegative decimal numbers representable in IEEE 754's double-precision (64-bit) floating-point format.
- Character literals are enclosed in single quotation marks, such as `'c'`.
- String literals are enclosed in double quotation marks, such as `"compiler"`.

Escape characters The following characters in character and string literals should be escaped with a backslash:

character	escape sequence
backslash (\)	`\\`
single quotation (')	`\'`
double quotation (")	`\"`
newline	`\n`
carriage return	`\r`
tab character	`\t`
backspace character	`\b`

It is a compile-time error if other escape sequences are encountered.

6.2 Function applications

No parentheses are necessary when applying a function to arguments. Instead, we can simply write the name of the function, followed by argument expressions. If any argument is a compound expression, the argument should still be enclosed in parentheses, such as `fib (2+3)`. Without the parentheses, `fib 2+3` would be parsed as `(fib 2)+3`.

Because we need not use parentheses to enclose the function arguments, it is possible to *partially apply* a function, without providing the complete list of arguments. For example, we can partially apply the `map` function by providing only the transformation function but not the list to be mapped with:

```
1 let partMap = map parseInt
2 in partMap ("261" :: "406" :: [])
```

6.3 Arithmetic expressions

Our language supports the usual binary arithmetic operations: addition (`+`), subtraction (`-`), multiplication (`*`), division (`/`), integer division (`//`), and integer modulus (`%`). Unary negation (`-`) is also supported.

6.4 Comparison expressions

Our language support the usual binary arithmetic comparisons: less than (`<`), less than or equal to (`<=`), greater than or equal to (`>=`), and greater than (`>`). General binary comparisons, including equals (`==`) and not equals (`!=`), are also supported.

6.5 Boolean expressions

Binary boolean operations (`&&` and `||`) are built-in and can be approximately defined in our language as follows:

```
1 def (&&) (p : Bool) (q : Bool) : Bool =
2   match (p, q) with
3   | (True, True) -> True
4   | _ -> False
5
6 def (||) (p : Bool) (q : Bool) : Bool =
7   match (p, q) with
8   | (False, False) -> False
9   | _ -> True
```

Boolean negation can be defined within our language, as shown in function `not` in Listing 3.

6.6 Pattern matching

Pattern matching is a mechanism for taking apart a compound value, such as one created by a user-defined data constructor. A **`match`** expression can be used for pattern matching. The value to be pattern-matched follows the **`match`** keyword, and cases follow the **`with`** keyword. Each case specifies a pattern, along with the resulting expression if the value matches the pattern. For instance, we can pattern-match a list value `l` as follows:

```
match l with
| [] -> True
| hd::tl -> False
```

The expression above simply determines whether `l` is the empty list. In the second case (`hd::tl`), the pattern contains bindings `hd` that refers to the first element of `l`, and `tl` that refers to the remaining elements of `l`. Pattern bindings can be used in their respective expression that follows.

The execution of a **`match`** statement proceeds from top to bottom, returning the expression corresponding to the first pattern that matches the value. Later patterns that could have matched the value are skipped.

Patterns can be nested, as in the following example:

```
match l with
| [] -> True
| x1::x2::xs -> x1 == x2
| hd::tl -> False
```

Here, the second pattern further deconstructs the tail of the list into nonempty sublist. This match expression determines whether `l` is empty, or has at least two elements and the first two elements are equal.

Wildcard patterns In the example above, the `hd` and `tl` bindings are never used in the resulting expression; there's no need to declare them in the first place. In such cases, we can use the *wildcard pattern* instead by writing `_`:

```
match l with
| [] -> True
| _::_ -> False
```

Better yet, since list values that can match the second case can only be nonempty lists, there's no need to specify the shape of a nonempty list; we can simplify the pattern as follows:

```
match l with
| [] -> True
| _ -> False
```

As an extension, your compiler may issue warnings on unused variables; this is not required.

Pattern redundancy Since patterns are evaluated from top to bottom, it is possible that a later pattern will never be matched, as in the following example:

```
match l with
| [] -> True
| hd::tl -> False
| x1::x2::xs -> x1 == x2
```

Here, the third pattern is never matched, because any list of length at least two will have matched the second pattern. In this case, the third pattern is *redundant*: one can safety remove the entire case without altering the program behavior.

As an extension, your compiler may issue warnings on redundant pattern cases; this is not required.

Nonexhaustive pattern matching It is possible that a compound value does not match any of the patterns, especially when cases are missing, as in the following example:

```
match l with
| [] -> True
```

Here, a nonempty list will not match the expression above. In this case, the **`match`** expression is *nonexhaustive*. A run-time error occurs if a value does not match any of the patterns provided.

As an extension, your compiler may issue warnings on nonexhaustive pattern matching expressions; this is not required.

6.7 `let` bindings

Local variables and functions can be declared within ***`let`*** *bindings*. The syntax for a local variable binding is **`let`** `x = e1` **`in`** `e2`, where `e1` is the definition of the binding and `e2` is to body of the **`let`** expression. The syntax for a local function binding is similar to that of top-level functions, but the **`def`** keyword is replaced by **`let`**.

Local variables are identifiers starting with a lowercase letter. A compile-time error occurs if one attempts to name a local variable starting with an uppercase letter.

Mutually recursive local functions are allowed in a **`let`** binding. In such cases, use the **`and`** keyword to separate the definitions, as in this example, which computes the Hofstadter Female and Male sequences:

```
1  def hofstadter (n : Integer) : (Integer, Integer) =
2    let f (n : Integer) : Integer =
3          if n == 0 then 1 else n - m (f (n-1))
4    and m (n : Integer) : Integer =
5          if n == 0 then 0 else n - f (m (n-1))
6    in (f n, m n)
```

Using the **`and`** keyword to declare multiple local variables is also permitted.

The local variable bindings introduced in a **`let`** expression is in scope only in the body. If multiple local variables are declared within the same **`let`** binding, the definition of a later variable cannot refer to an earlier binding. In such cases, use a nested **`let`** in the body.

On the other hand, the local function bindings introduced in a **`let`** expression is in scope in both the function definition and the body. As a result, it is possible to define a nonterminating function like this:

```
def bot () : Integer =
  let forever (x : Integer) : Integer = forever x
  in forever 42
```

As an extension, your compiler may issue warnings on (obviously) nonterminating recursion (both in **`def`** and **`let`**); this is not required.

As an extension, your compiler may issue warnings on unused local variables and functions declared in **`let`** bindings; this is not required.

6.8 Lambda expressions

Local, nonrecursive functions may be declared in another way using *lambda expressions*, a form of anonymous functions. A lambda expression begins with a backslash symbol (`\`), followed by one or more parameters of the function, a colon (`:`), the return type, a double-arrow (`=>`), and the function body. The scope of the parameters is the function body.

As an extension, your compiler may accept lambda expressions that do not specify parameter and return types. Your compiler will need to perform *type inference* on variables and expressions; this is not required.

6.9 Conditional expressions

The conditional expression **`if`** `e` **`then`** `e1` **`else`** `e2` has the same meaning as `e ? e1 : e2` in Java-like languages.

6.10 Precedence and associativity

Precedence and associativity of operators and expressions are defined as follows, from higher to lower precedence.

operation/expression	associativity
function application	left-associative
function composition (`.`)	right-associative
unary negation (`-`)	–
`*`, `/`, `//`, `%`	left-associative
`+`, `-`	left-associative
`::`, `++`	right-associative
`==`, `!=`, `<`, `<=`, `>=`, `>`	nonassociative
`&&`	right-associative
`||`	right-associative
`$`	right-associative
`let`, **`match`**, conditional, lambda expressions	–

7 Type system

(This section is under construction.)

7.1 Expression typing rules

These typing rules are for validating expressions in a CPE program. Judgment $\Gamma \vdash e : \tau$ means that in a typing context Γ, expression e has type τ. Literal rules are obvious and hence omitted.

$$\frac{\Gamma(x) = \tau}{\Gamma \vdash x : \tau}\ \textsc{var} \qquad \frac{\Gamma \vdash e_1 : \text{Integer} \qquad \Gamma \vdash e_2 : \text{Integer} \qquad \oplus \in \{+, -, *, //, \%\}}{\Gamma \vdash e_1 \oplus e_2 : \text{Integer}}\ \textsc{arith-int}$$

$$\frac{\Gamma \vdash e_1 : \text{Integer} \qquad \Gamma \vdash e_2 : \text{Integer}}{\Gamma \vdash e_1 / e_2 : \text{Double}}\ \textsc{arith-int-div}$$

$$\frac{\Gamma \vdash e : \tau \qquad \tau \in \{\text{Integer}, \text{Double}\}}{\Gamma \vdash -e : \tau}\ \textsc{arith-neg}$$

$$\frac{\Gamma \vdash e_1 : \text{Double} \qquad \Gamma \vdash e_2 : \text{Double} \qquad \oplus \in \{+, -, *, /\}}{\Gamma \vdash e_1 \oplus e_2 : \text{Double}}\ \textsc{arith-double}$$

$$\frac{\Gamma \vdash e_1 : \text{Integer} \qquad \Gamma \vdash e_2 : \text{Double} \qquad \oplus \in \{+, -, *, /\}}{\Gamma \vdash e_1 \oplus e_2 : \text{Double}}\ \textsc{arith-coerce-l}$$

$$\frac{\Gamma \vdash e_1 : \text{Double} \qquad \Gamma \vdash e_2 : \text{Integer} \qquad \oplus \in \{+, -, *, /, \%\}}{\Gamma \vdash e_1 \oplus e_2 : \text{Double}}\ \textsc{arith-coerce-r}$$

$$\frac{\Gamma \vdash e_1 : \text{Double} \qquad \Gamma \vdash e_2 : \text{Integer}}{\Gamma \vdash e_1 // e_2 : \text{Integer}}\ \textsc{arith-double-div}$$

$$\frac{\Gamma \vdash e_1 : \tau \qquad \Gamma \vdash e_2 : \tau \qquad \oplus \in \{==, !=, <, <=, >=, >\} \qquad \tau \in \{\text{Integer}, \text{Double}\}}{\Gamma \vdash e_1 \oplus e_2 : \text{Bool}}\ \textsc{numeric-comp}$$

$$\frac{\Gamma \vdash e_1 : \alpha \qquad \Gamma \vdash e_2 : \alpha \qquad \oplus \in \{==, !=\}}{\Gamma \vdash e_1 \oplus e_2 : \text{Bool}}\ \textsc{comp}$$

$$\frac{\Gamma \vdash e_1 : \text{Bool} \qquad \Gamma \vdash e_2 : \text{Bool} \qquad \oplus \in \{\texttt{\&\&}, \texttt{||}\}}{\Gamma \vdash e_1 \oplus e_2 : \text{Bool}}\ \textsc{bool}$$

$$\frac{\Gamma \vdash e_1 : \alpha \qquad \Gamma \vdash e_2 : [\alpha]}{\Gamma \vdash e_1 :: e_2 : [\alpha]}\ \textsc{list-cons} \qquad \frac{\Gamma \vdash e_1 : [\alpha] \qquad \Gamma \vdash e_2 : [\alpha]}{\Gamma \vdash e_1 \texttt{++} e_2 : [\alpha]}\ \textsc{list-concat}$$

$$\frac{\Gamma \vdash f : \alpha \to \beta \qquad \Gamma \vdash x : \alpha}{\Gamma \vdash f\ x : \beta}\ \textsc{func-app-l} \qquad \frac{\Gamma \vdash f : \alpha \to \beta \qquad \Gamma \vdash x : \alpha}{\Gamma \vdash f\ \texttt{\$}\ x : \beta}\ \textsc{func-app-r}$$

$$\frac{\Gamma \vdash f : \alpha \to \beta \qquad \Gamma \vdash g : \beta \to \gamma}{\Gamma \vdash g\ .\ f : \alpha \to \gamma}\ \textsc{func-comp}$$

$$\frac{\Gamma \vdash e_1 : \text{Bool} \qquad \Gamma \vdash e_2 : \alpha \qquad \Gamma \vdash e_3 : \alpha}{\Gamma \vdash \texttt{if}\ e_1\ \texttt{then}\ e_2\ \texttt{else}\ e_3 : \alpha}\ \textsc{cond}$$

$$\frac{\Gamma, x_1 : \alpha_1, x_2 : \alpha_2, \ldots, x_n : \alpha_n \vdash e : \beta}{\Gamma \vdash (\backslash (x_1 : \alpha_1)(x_2 : \alpha_2) \cdots (x_n : \alpha_n) \colon \beta \texttt{ => } e) : \alpha_1 \to \alpha_2 \to \cdots \to \alpha_n \to \beta}\ \textsc{lambda-decl}$$

$$\frac{\begin{array}{c}\Gamma \vdash e_1 : \alpha_1 \\ \vdots \\ \Gamma \vdash e_n : \alpha_n \\ \Gamma, x_1 : \alpha_1, \ldots, x_n : \alpha_n \vdash e : \beta\end{array}}{\Gamma \vdash \texttt{let}\ x_1 = e_1\ \texttt{and}\ \cdots\ \texttt{and}\ x_n = e_n\ \texttt{in}\ e : \beta}\ \textsc{let-var}$$

$$\frac{\begin{array}{c}\Gamma, x_{11} : \alpha_{11}, \ldots, x_{1m_1} : \alpha_{1m_1}, (x_1 : \alpha_{11} \to \cdots \to \alpha_{1m_1} \to \beta_1), \ldots \vdash e_1 : \beta_1 \\ \vdots \\ \Gamma, x_{n1} : \alpha_{n1}, \ldots, x_{nm_n} : \alpha_{nm_n}, (x_1 : \alpha_{11} \to \cdots \to \alpha_{1m_1} \to \beta_1), \ldots \vdash e_n : \beta_n \\ \Gamma, (x_1 : \alpha_{11} \to \cdots \to \alpha_{1m_1} \to \beta_1), \ldots \vdash e : \beta\end{array}}{\Gamma \vdash \left(\texttt{let}\ x_1\ (x_{11} : \alpha_{11}) \cdots (x_{1n_1} : \alpha_{1m_1}) \colon \beta_1 = e_1\ \texttt{and}\ \cdots\ \texttt{in}\ e\right) : \beta}\ \textsc{let-func}$$

$$\frac{\begin{array}{c}\Gamma \vdash e : \alpha \\ \Gamma, ? \vdash e_1 : \beta \\ \vdots \\ \Gamma, ? \vdash e_n : \beta\end{array}}{\Gamma \vdash \texttt{match}\ e\ \texttt{with}\ \texttt{|}\ p_1\ \texttt{->}\ e_1\ \cdots\ \texttt{|}\ p_n\ \texttt{->}\ e_n : \beta}\ \textsc{match}$$

7.2 Type expression typing rules

These typing rules are for validating type expressions in a CPE program. Judgment $\Gamma \vdash \tau$ means that in a typing context Γ, type τ is well-formed.

7.3 Pattern typing rules

These typing rules are for validating patterns in a CPE program. Judgment ??? means that ???.